Keras課堂實驗(二)

乳癌資料

高怡宣
陽明大學 生物醫學影像暨放射科學系

Data: Breast Cancer Wisconsin Data Set, Predict cancer is benign or malignant
https://www.kaggle.com/uciml/breast-cancer-wisconsin-data

Code: Feature Selection and Data Visualization
https://www.kaggle.com/kanncaa1/feature-selection-and-data-visualization

kaggle | Search kaggle | Competitions | Datasets | Kernels | Discussion | Learn | ••• | Sign In

Dataset

Breast Cancer Wisconsin (Diagnostic) Data Set

Predict whether the cancer is benign or malignant

496 voters

share

UCI ML UCI Machine Learning • updated 2 years ago (Version 2)

Data | Overview | Kernels | Discussion | Activity | Download (48 KB) | New Kernel

Data (48 KB) | API | kaggle datasets download -d uciml/breast-cancer-… | ? | Download All

Data Sources

data.csv 32 columns

About this file

This file contains all the data in

Columns

id ID number

diagnosis The diagnosis of breast

https://www.kaggle.com/uciml/breast-cancer-wisconsin-data

kaggle Search kaggle Competitions Datasets Kernels Disc

DATAI

DATAI

Feature Selection and Data Visualization

last run 4 months ago · IPython Notebook HTML · 49,228 views
using data from Breast Cancer Wisconsin (Diagnostic) Data Set

Notebook Code Data (1) Comments (195) Log Versions (514) Forks (880)

You are viewing the last successful run of this kernel. Click here to see the current version with an error.

Tags data visualization classification feature engineering healthcare

https://www.kaggle.com/kanncaa1/feature-selection-and-data-visualization

jupyter Keras_02_Breast_Cancer Last Checkpoint: 1 分鐘前 (unsaved changes)

Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3

Run Markdown

Breast Cancer Wisconsin (Diagnostic) Data Set, Predict whether the cancer is benign or malignant

data from: https://www.kaggle.com/uciml/breast-cancer-wisconsin-data

code modified from: https://www.kaggle.com/kanncaa1/feature-selection-and-data-visualization

INTRODUCTION

In this data analysis report, I usually focus on feature visualization and selection as a different from other kernels. Feature selection with correlation, univariate feature selection, recursive feature elimination, recursive feature elimination with cross validation and tree based feature selection methods are used with random forest classification. Apart from these, principle component analysis are used to observe number of components.

Enjoy your data analysis!!!

Data Analysis

In [1]:
```
# This Python 3 environment comes with many helpful analytics librarie
# It is defined by the kaggle/python docker image: https://github.com/
# For example, here's several helpful packages to load in

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import seaborn as sns # data visualization library
import matplotlib.pyplot as plt
# Input data files are available in the "../input/" directory.
# For example, running this (by clicking run or pressing Shift+Enter)
import time
from subprocess import check_output
#print(check_output(["ls", "../input"]).decode("utf8"))

# Any results you write to the current directory are saved as output.
```

In [2]:
```
data = pd.read_csv('../input/data.csv')
```

Before making anything like feature selection,feature extraction and classification, firstly look at features of data.

In [3]:
```
data.head()  # head method show only first 5 rows
```

Out[3]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smo
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 33 columns

Reminder

```
In [3]: iris = pd.read_csv("../input/Iris.csv")
```

```
In [ ]: iris.
```

```
In [4]: iris.head()  # .head gives the first 5 rows
```

按 Tab, 找DataFrame的用法

Reminder

```
argument of issubdtype from `float` to `np.floating` is deprecated.
float).type`.
gister_converters as _register_converters
l.
```

```
iris.Id
iris.PetalLengthCm
iris.PetalWidthCm
iris.SepalLengthCm
iris.SepalWidthCm
iris.Species
iris.T
iris.abs
iris.add
iris.add_prefix
```

In [2]:

In [3]: `nput/Iris.csv")`

In [ ]: `iris.`

In [4]:
```
iris.head()  # .head gives the first 5 rows
```

```python
# feature names as a list
col = data.columns        # .columns gives columns names in data
print(col)
```

```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32'],
      dtype='object')
```

```
In [5]: data.isnull().sum()
```

```
Out[5]: id                          0
        diagnosis                   0
        radius_mean                 0
        texture_mean                0
        perimeter_mean              0
        area_mean                   0

        symmetry_worst              0
        fractal_dimension_worst     0
        Unnamed: 32               569
        dtype: int64
```

There are 4 things that take my attention 1) There is an **id** that cannot be used for classificaiton 2) **Diagnosis** is our class label 3) **Unnamed: 32** feature includes NaN so we do not need it. 4) I do not have any idea about other feature names actually I do not need because machine learning is awesome **:)**

Therefore, drop these unnecessary features. However do not forget this is not a feature selection. This is like a browse a pub, we do not choose our drink yet !!!

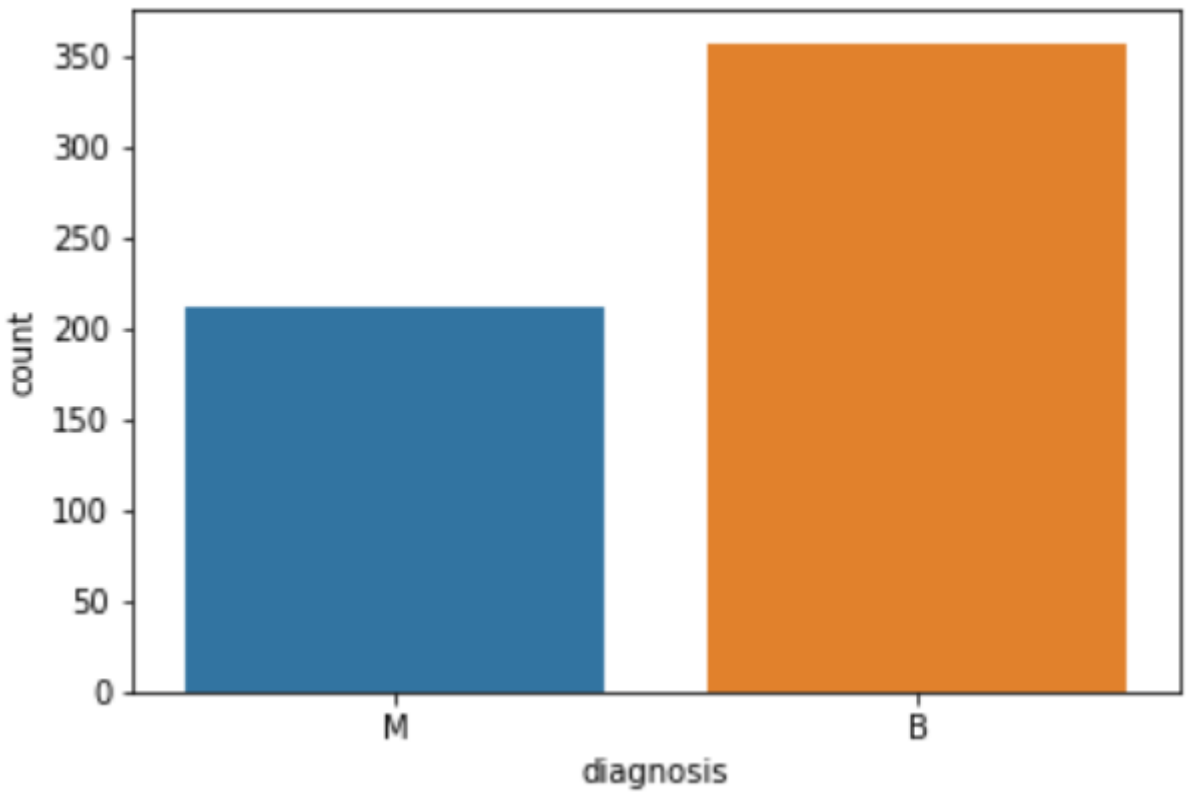
```python
# y includes our labels and x includes our features
y = data.diagnosis                                  # M or B
list = ['Unnamed: 32','id','diagnosis']
x = data.drop(list,axis = 1 )
x.head()
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean
0	17.99	10.38	122.80	1001.0	0.11840	0.27760
1	20.57	17.77	132.90	1326.0	0.08474	0.07864
2	19.69	21.25	130.00	1203.0	0.10960	0.15990
3	11.42	20.38	77.58	386.1	0.14250	0.28390
4	20.29	14.34	135.10	1297.0	0.10030	0.13280

5 rows × 30 columns

```python
ax = sns.countplot(y,label="Count")        # M = 212, B = 357
print(y.value_counts())
l1, l2 = y.value_counts()
print('Number of Benign: ',l1)
print('Number of Malignant : ',l2)
```

```
B    357
M    212
Name: diagnosis, dtype: int64
Number of Benign:  357
Number of Malignant :  212
```



```
x.describe()
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	co
count	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	14.127292	19.289649	91.969033	654.889104	0.096360	
std	3.524049	4.301036	24.298981	351.914129	0.014064	
min	6.981000	9.710000	43.790000	143.500000	0.052630	
25%	11.700000	16.170000	75.170000	420.300000	0.086370	
50%	13.370000	18.840000	86.240000	551.100000	0.095870	
75%	15.780000	21.800000	104.100000	782.700000	0.105300	
max	28.110000	39.280000	188.500000	2501.000000	0.163400	

8 rows × 30 columns

Visualization

In order to visualizate data we are going to use seaborn plots that is not used in other kernels to inform you and for diversity of plots. What I use in real life is mostly violin plot and swarm plot. Do not forget we are not selecting feature, we are trying to know data like looking at the drink list at the pub door.

Before violin and swarm plot we need to normalization or standirdization. Because differences between values of features are very high to observe on plot. I plot features in 3 group and each group includes 10 features to observe better.

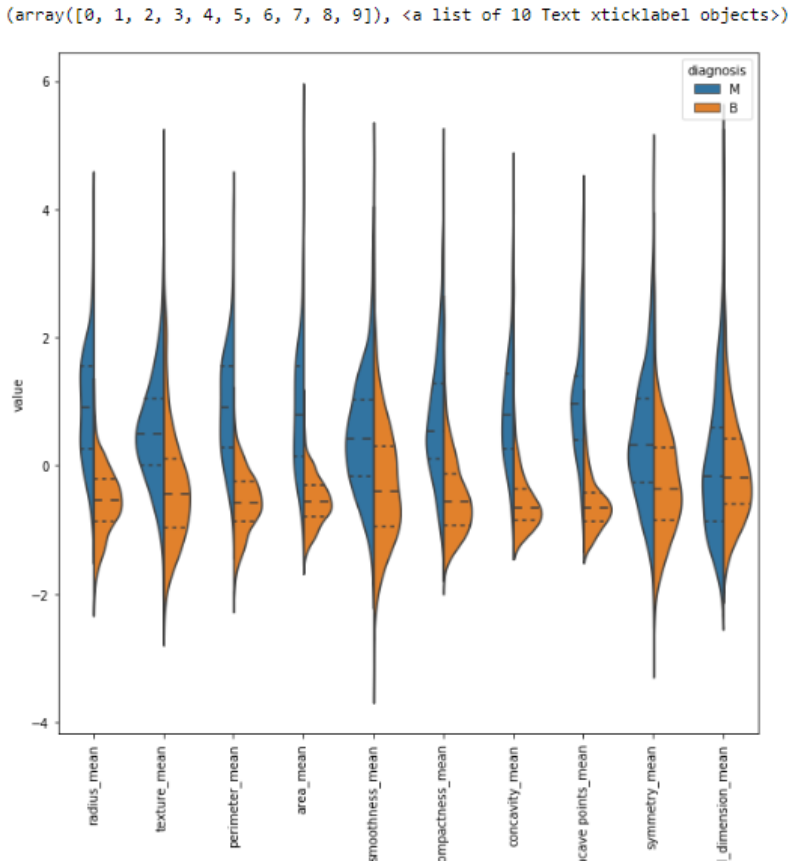
```
# first ten features
data_dia = y
data = x
data_n_2 = (data - data.mean()) / (data.std())      # standardization x only
data = pd.concat([y,data_n_2.iloc[:,0:10]],axis=1)  # combine x and y
print(data.shape)
data = pd.melt(data,id_vars="diagnosis",            # for violin plot
                    var_name="features",
                    value_name='value')
print(data.shape)
print(data)
```

```
(569, 11)
(5690, 3)
     diagnosis                 features     value
0            M             radius_mean  1.096100
1            M             radius_mean  1.828212
2            M             radius_mean  1.578499
3            M             radius_mean -0.768233
4            M             radius_mean  1.748758
5            M             radius_mean -0.475956
```

```
5686         M  fractal_dimension_mean -1.057681
5687         M  fractal_dimension_mean -0.894800
5688         M  fractal_dimension_mean  1.042778
5689         B  fractal_dimension_mean -0.560539

[5690 rows x 3 columns]
```

```
plt.figure(figsize=(10,10))
sns.violinplot(x="features", y="value", hue="diagnosis",\
               data=data,split=True, inner="quart")
plt.xticks(rotation=90)
```



seaborn.violinplot

`seaborn.violinplot` (*x=None, y=None, hue=None, data=None, order=None, hue_order=None, bw='scott', cut=2, scale='area', scale_hue=True, gridsize=100, width=0.8, inner='box', split=False, dodge=True, orient=None, linewidth=None, color=None, palette=None, saturation=0.75, ax=None, **kwargs*)

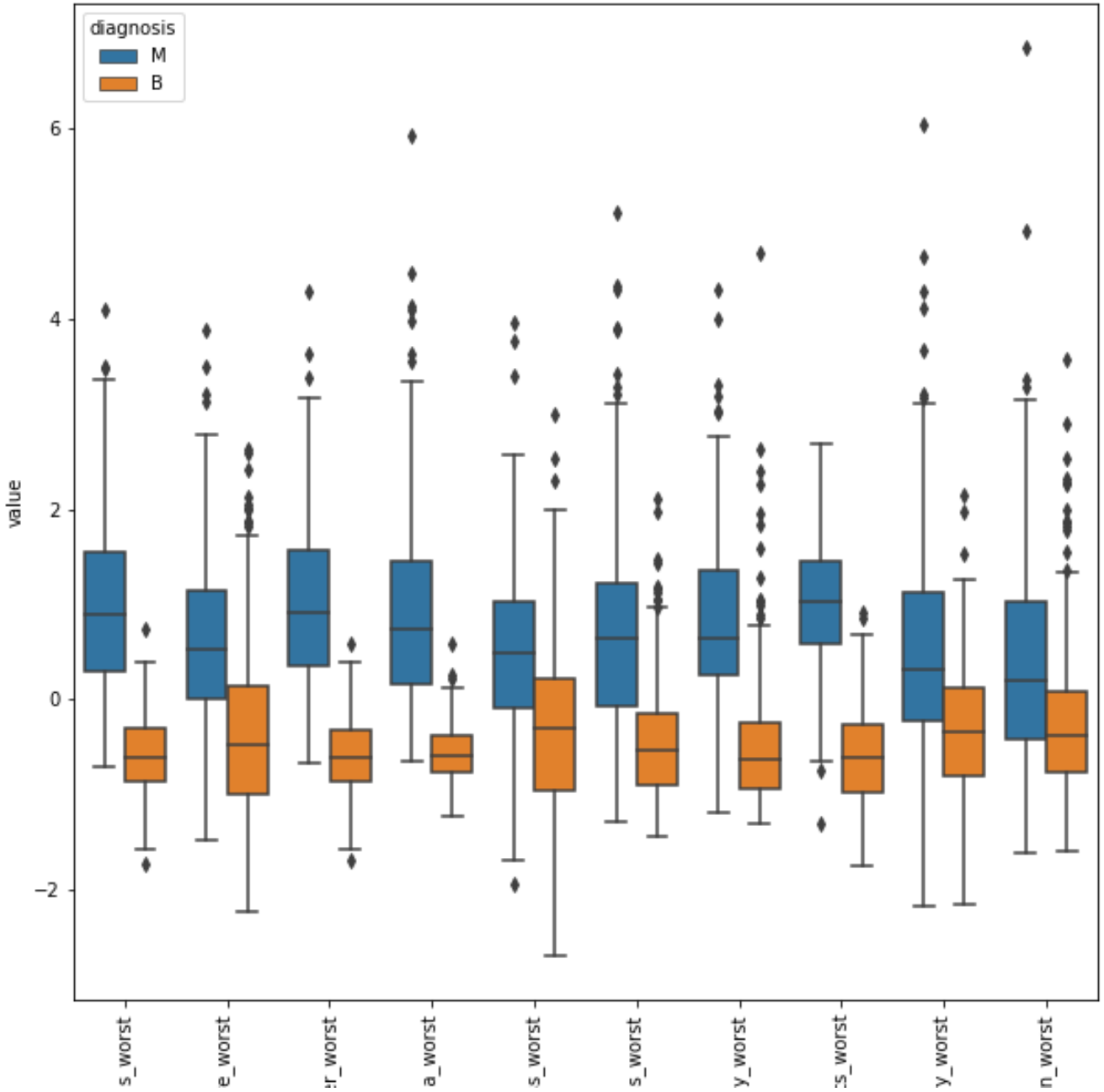
Draw a combination of boxplot and kernel density estimate.

A violin plot plays a similar role as a box and whisker plot. It shows the distribution of quantitative data across several levels of one (or more) categorical variables such that those distributions can be compared. Unlike a box plot, in which all of the plot components correspond to actual datapoints, the violin plot features a kernel density estimation of the underlying distribution.

```
# Second ten features
data = pd.concat([y,data_n_2.iloc[:,10:20]],axis=1)
data = pd.melt(data,id_vars="diagnosis",
                    var_name="features",
                    value_name='value')
plt.figure(figsize=(10,10))
sns.violinplot(x="features", y="value", hue="diagnosis", \
               data=data,split=True, inner="quart")
plt.xticks(rotation=90)
```

```
# last eleven features
data = pd.concat([y,data_n_2.iloc[:,20:31]],axis=1)
data = pd.melt(data,id_vars="diagnosis",
                    var_name="features",
                    value_name='value')
plt.figure(figsize=(10,10))
sns.violinplot(x="features", y="value", hue="diagnosis", \
               data=data,split=True, inner="quart")
plt.xticks(rotation=90)
```

```
# As an alternative of violin plot, box plot can be used
# box plots are also useful in terms of seeing outliers
# I do not visualize all features with box plot
# In order to show you lets have an example of box plot
# If you want, you can visualize other features as well.
plt.figure(figsize=(10,10))
sns.boxplot(x="features", y="value", hue="diagnosis", data=data)
plt.xticks(rotation=90)
```


diagnosis
M
B
value
6
4
2
0
−2
s_worst
e_worst
r_worst
a_worst
s_worst
s_worst
y_worst
s_worst
y_worst
n_worst

seaborn 0.9.0

seaborn.boxplot (*x=None, y=None, hue=None, data=None, order=None, hue_order=None, orient=None, color=None, palette=None, saturation=0.75, width=0.8, dodge=True, fliersize=5, linewidth=None, whis=1.5, notch=False, ax=None, **kwargs*)

Draw a box plot to show distributions with respect to categories.

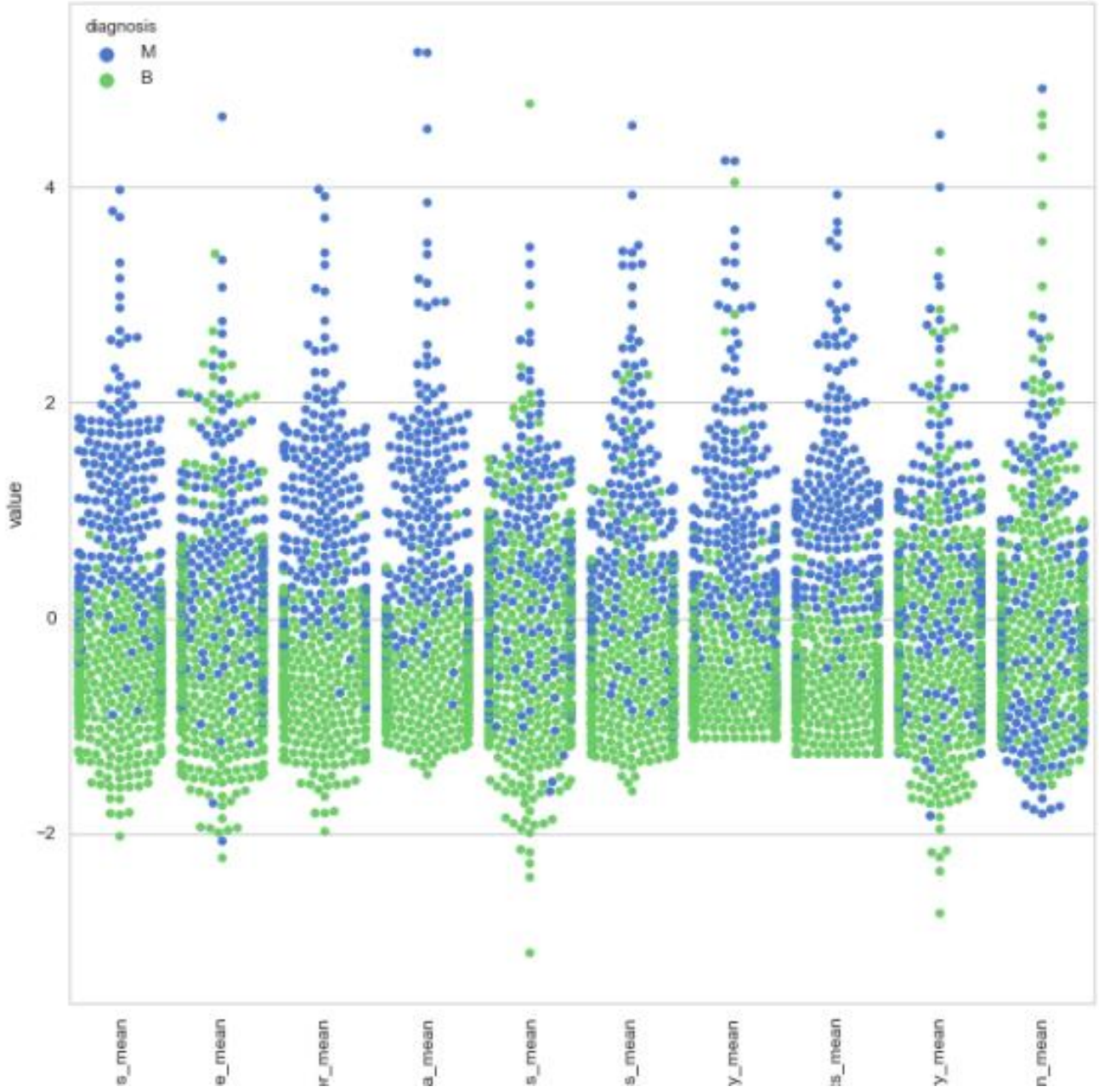
A box plot (or box-and-whisker plot) shows the distribution of quantitative data in a way that facilitates comparisons between variables or across levels of a categorical variable. The box shows the quartiles of the dataset while the whiskers extend to show the rest of the distribution, except for points that are determined to be “outliers” using a method that is a function of the inter-quartile range.

In swarm plot, I will do three part like violin plot not to make plot very complex appearance

In [15]:
```
sns.set(style="whitegrid", palette="muted")
data_dia = y
data = x
data_n_2 = (data - data.mean()) / (data.std())      # standardization
data = pd.concat([y,data_n_2.iloc[:,0:10]],axis=1)
data = pd.melt(data,id_vars="diagnosis",
                    var_name="features",
                    value_name='value')
plt.figure(figsize=(10,10))
tic = time.time()
sns.swarmplot(x="features", y="value", hue="diagnosis", data=data)

plt.xticks(rotation=90)
```

```
Out[15]: (array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]), <a list of 10 Text xticklabel objects>)
```



seaborn.swarmplot

`seaborn.swarmplot` (*x=None, y=None, hue=None, data=None, order=None, hue_order=None, dodge=False, orient=None, color=None, palette=None, size=5, edgecolor='gray', linewidth=0, ax=None, **kwargs*) ¶

Draw a categorical scatterplot with non-overlapping points.

This function is similar to `stripplot()`, but the points are adjusted (only along the categorical axis) so that they don't overlap. This gives a better representation of the distribution of values, but it does not scale well to large numbers of observations. This style of plot is sometimes called a "beeswarm".

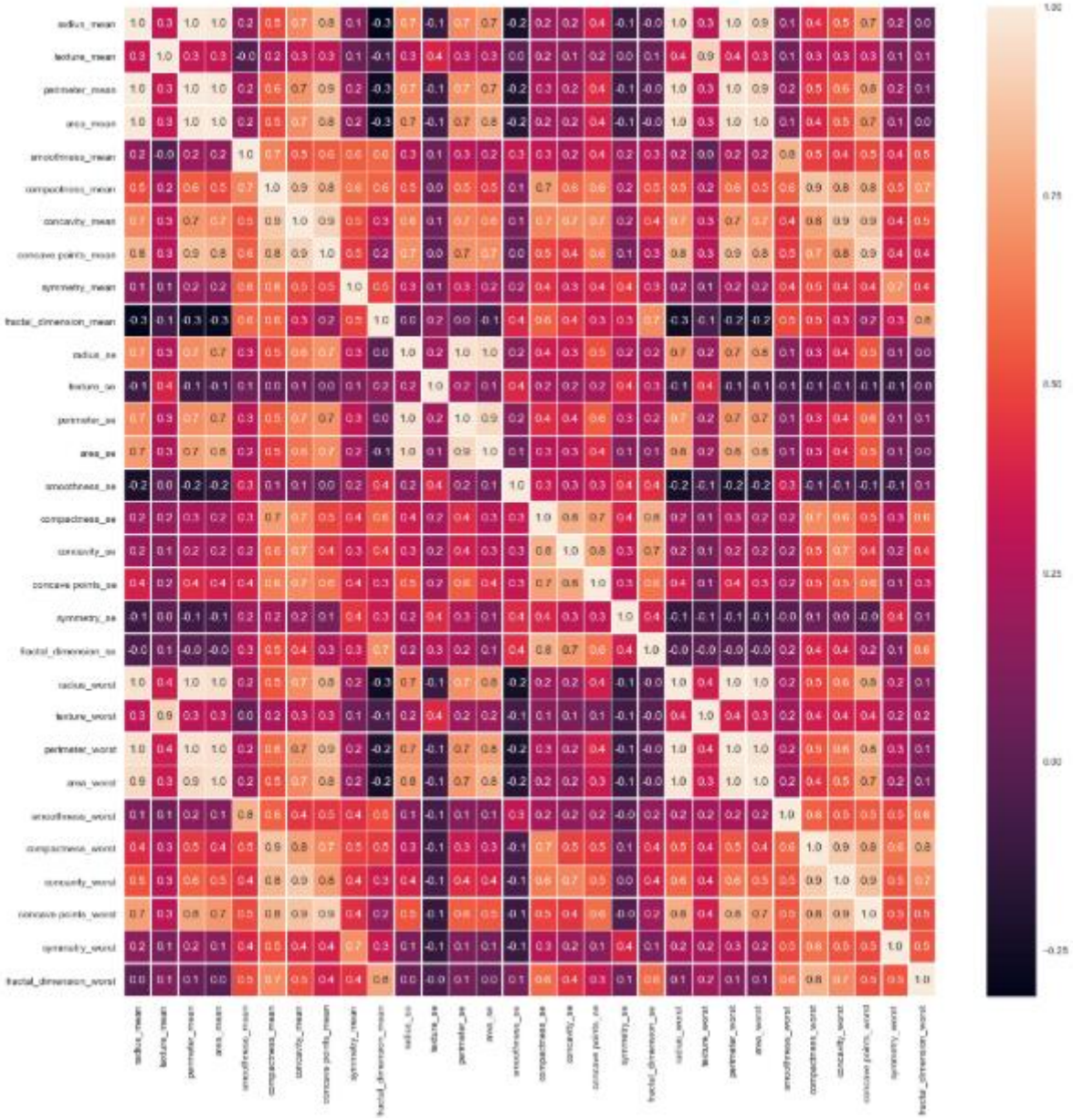
Homework:

(1) Change Swarmplot to stripplot
(2) Replace RandomForestClassifier by sequential ANN
(3) Increase accuracy to 98%
(4) Add confusion matrix to Iris codes

What if we want to observe all correlation between features? Yes, you are right. The answer is heatmap that is old but powerful plot method.

In [18]:
```
#correlation map
f,ax = plt.subplots(figsize=(18, 18))
sns.heatmap(x.corr(), annot=True, linewidths=.5, fmt= '.1f',ax=ax)
```

Out[18]:
```
<matplotlib.axes._subplots.AxesSubplot at 0xd9fe400>
```


radius_mean
texture_mean
perimeter_mean
area_mean
smoothness_mean
compactness_mean
concavity_mean
concave points_mean
symmetry_mean
fractal_dimension_mean
radius_se
texture_se
perimeter_se
area_se
smoothness_se
compactness_se
concavity_se
concave points_se
symmetry_se
fractal_dimension_se
radius_worst
texture_worst
perimeter_worst
area_worst
smoothness_worst
compactness_worst
concavity_worst
concave points_worst
symmetry_worst
fractal_dimension_worst
1.00
0.75
0.50
0.25
0.00
-0.25

Feature Selection and Random Forest Classification

Today our purpuse is to try new cocktails. For example, we are finaly in the pub and we want to drink different tastes. Therefore, we need to compare ingredients of drinks. If one of them includes lemon, after drinking it we need to eliminate other drinks which includes lemon so as to experience very different tastes.

In this part we will select feature with different methods that are feature selection with correlation, univariate feature selection, recursive feature elimination (RFE), recursive feature elimination with cross validation (RFECV) and tree based feature selection. We will use random forest classification in order to train our model and predict.

1) Feature selection with correlation and random forest classification

As it can be seen in map heat figure **radius_mean, perimeter_mean and area_mean** are correlated with each other so we will use only **area_mean**. If you ask how i choose **area_mean** as a feature to use, well actually there is no correct answer, I just look at swarm plots and **area_mean** looks like clear for me but we cannot make exact separation among other correlated features without trying. So lets find other correlated features and look accuracy with random forest classifier.

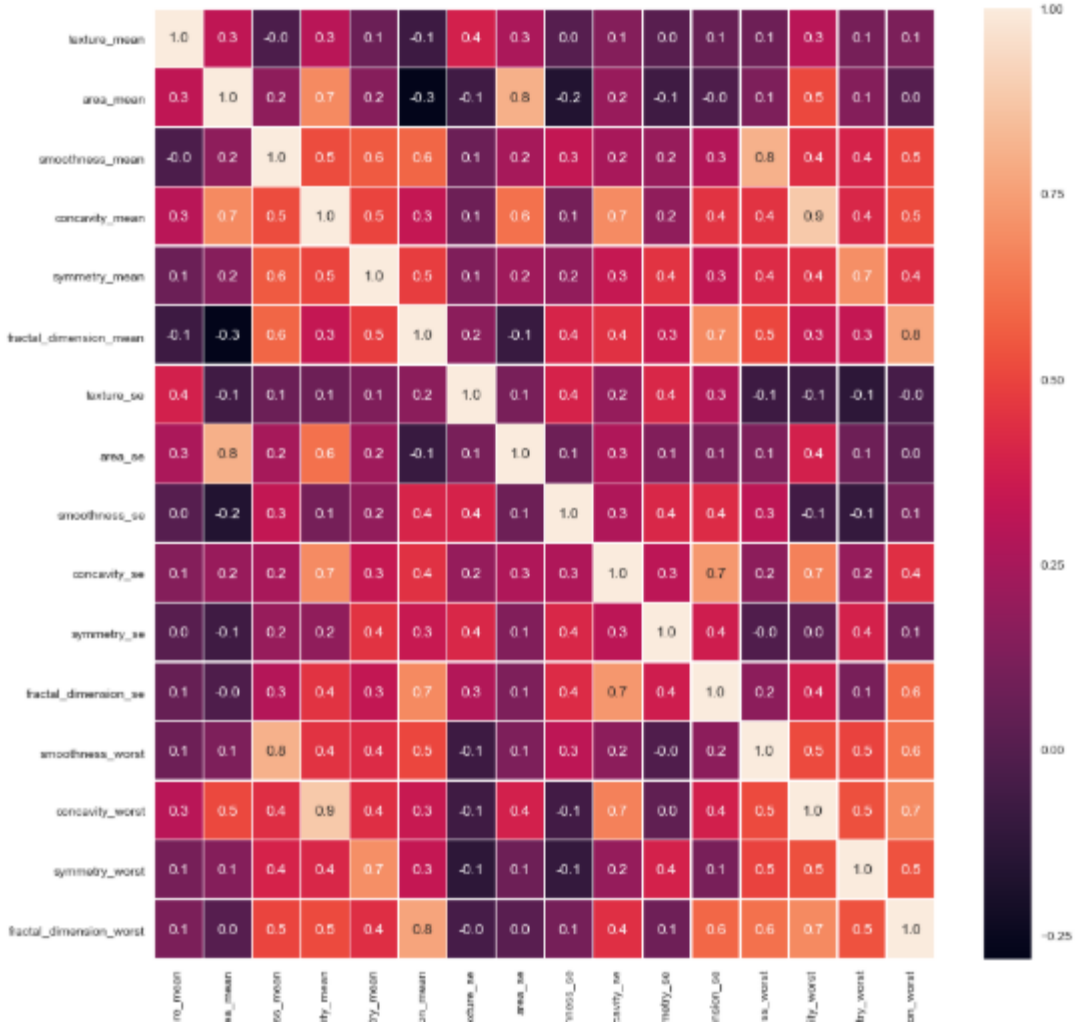
Compactness_mean, concavity_mean and concave points_mean are correlated with each other.Therefore I only choose **concavity_mean**. Apart from these, **radius_se, perimeter_se and area_se** are correlated and I only use **area_se**. **radius_worst, perimeter_worst and area_worst** are correlated so I use **area_worst**. **Compactness_worst, concavity_worst and concave points_worst** so I use **concavity_worst**. **Compactness_se, concavity_se and concave points_se** so I use **concavity_se**. **texture_mean and texture_worst are correlated** and I use **texture_mean**. **area_worst and area_mean** are correlated, I use **area_mean**.

```
drop_list1 = ['perimeter_mean','radius_mean','compactness_mean','concave points_mean',\
              'radius_se','perimeter_se','radius_worst','perimeter_worst',\
              'compactness_worst','concave points_worst','compactness_se',\
              'concave points_se','texture_worst','area_worst']

x_1 = x.drop(drop_list1,axis = 1 )        # do not modify x, we will use it later
x_1.head()
```

	texture_mean	area_mean	smoothness_mean	concavity_mean	symmetry_mean	fractal_dimension_mean
0	10.38	1001.0	0.11840	0.3001	0.2419	0.07871
1	17.77	1326.0	0.08474	0.0869	0.1812	0.05667
2	21.25	1203.0	0.10960	0.1974	0.2069	0.05999
3	20.38	386.1	0.14250	0.2414	0.2597	0.09744
4	14.34	1297.0	0.10030	0.1980	0.1809	0.05883

In [20]:
```
#correlation map
f,ax = plt.subplots(figsize=(14, 14))
sns.heatmap(x_1.corr(), annot=True, linewidths=.5, fmt= '.1f',ax=ax)
```



In [21]:
```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score,confusion_matrix
from sklearn.metrics import accuracy_score
```

```
# split data train 70 % and test 30 %
x_train, x_test, y_train, y_test = train_test_split(x_1, y, \
                                    test_size=0.3, random_state=42)
```

sklearn.model_selection.train_test_split

sklearn.model_selection. **train_test_split** (**arrays, **options*) [source]

Split arrays or matrices into random train and test subsets

Quick utility that wraps input validation and `next(ShuffleSplit().split(X, y))` and application to input data into a single call for splitting (and optionally subsampling) data in a oneliner.

Parameters: ***arrays : *sequence of indexables with same length / shape[0]***

Allowed inputs are lists, numpy arrays, scipy-sparse matrices or pandas dataframes.

test_size : *float, int or None, optional (default=0.25)*

If float, should be between 0.0 and 1.0 and represent the proportion of the dataset to include in the test split. If int, represents the absolute number of test samples. If None, the value is set to the complement of the train size. By default, the value is set to 0.25. The default will change in version 0.21. It will remain 0.25 only if `train_size` is unspecified, otherwise it will complement the specified `train_size`.

train_size : *float, int, or None, (default=None)*

If float, should be between 0.0 and 1.0 and represent the proportion of the dataset to include in the train split. If int, represents the absolute number of train samples. If None, the value is automatically set to the complement of the test size.

random_state : ***int, RandomState instance or None, optional (default=None)***

If int, random_state is the seed used by the random number generator; If RandomState instance, random_state is the random number generator; If None, the random number generator is the RandomState instance used by np.random.

shuffle : ***boolean, optional (default=True)***

Whether or not to shuffle the data before splitting. If shuffle=False then stratify must be None.

```
#random forest classifier with n_estimators=10 (default)
clf_rf = RandomForestClassifier(random_state=43)
clr_rf = clf_rf.fit(x_train,y_train)
```

sklearn.ensemble.RandomForestClassifier

class sklearn.ensemble. **RandomForestClassifier** (*n_estimators='warn', criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None*) [source]

A random forest classifier.

A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. The sub-sample size is always the same as the original input sample size but the samples are drawn with replacement if bootstrap=True (default).

Parameters: **n_estimators** : ***integer, optional (default=10)***

The number of trees in the forest.

Changed in version 0.20: The default value of `n_estimators` will change from 10 in version 0.20 to 100 in version 0.22.

criterion : ***string, optional (default="gini")***

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain. Note: this parameter is tree-specific.

max_depth : ***integer or None, optional (default=None)***

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

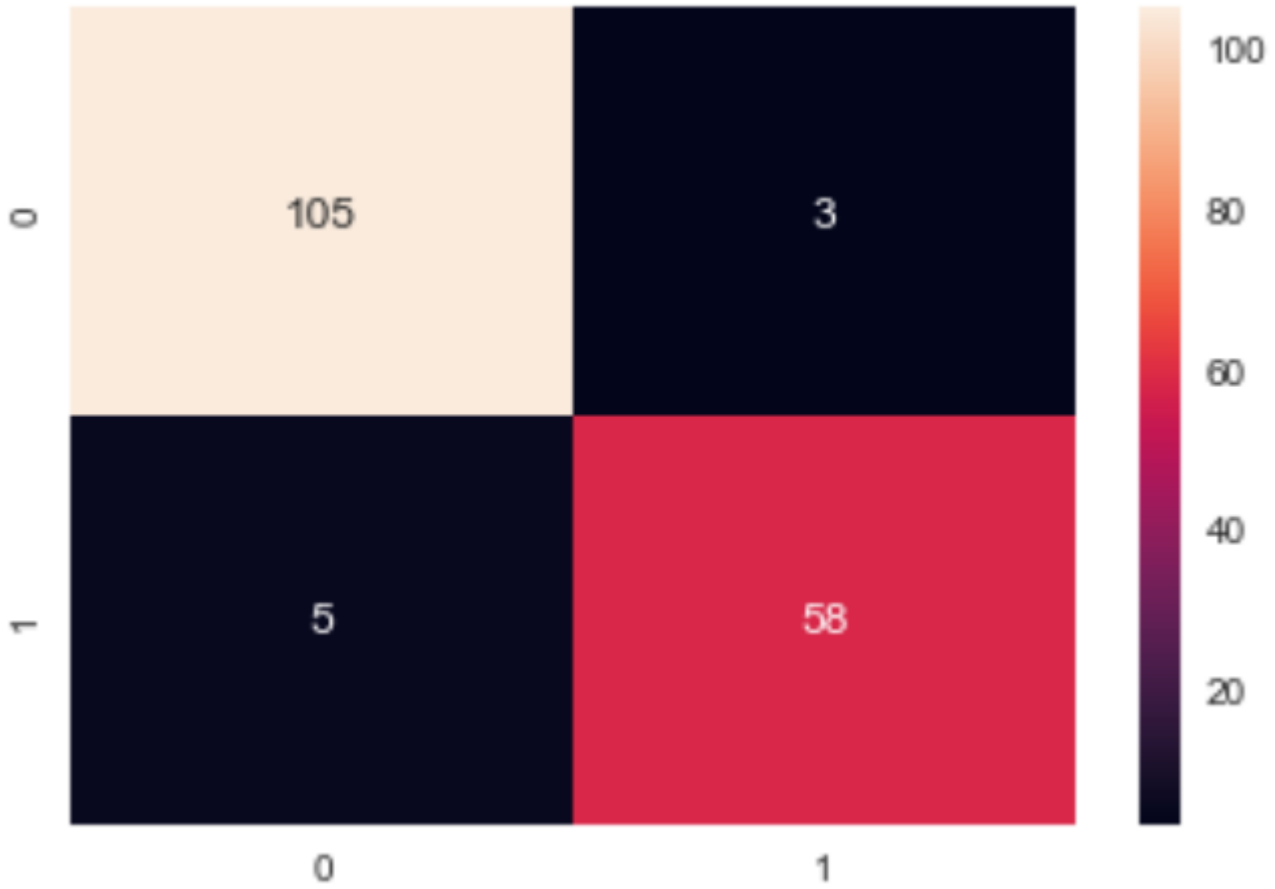
Homework - 2:

(5) Add cross validation

(6) Change to: clf_rf = RandomForestClassifier (n_estimators=10, max_depth=8, random_state=3)

```
ac = accuracy_score(y_test,clf_rf.predict(x_test))
print('Accuracy is: ',ac)
cm = confusion_matrix(y_test,clf_rf.predict(x_test))
sns.heatmap(cm,annot=True,fmt="d")
```

```
Accuracy is:  0.9532163742690059
```


105
3
5
58
0
1
0
1
100
80
60
40
20

2) Univariate feature selection and random forest classification

In univariate feature selection, we will use SelectKBest that removes all but the k highest scoring features. http://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html#sklearn.feature_sele

In this method we need to choose how many features we will use. For example, will k (number of features) be 5 or 10 or 15? The answer is only trying or intuitively. I do not try all combinations but I only choose k = 5 and find best 5 features.

In [22]:
```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
# find best scored 5 features
select_feature = SelectKBest(chi2, k=5).fit(x_train, y_train)
```

In [23]:
```
print('Score list:', select_feature.scores_)
print('Feature list:', x_train.columns)
```

Score list: [

6.06916433e+01
3.66899557e+04
1.00015175e-01
1.30547650e+01
1.95982847e-01
3.42575072e-04
4.07131026e-02
6.12741067e+03
1.32470372e-03
6.92896719e-01
1.39557806e-03
2.65927071e-03
2.63226314e-01
2.58858117e+01
1.00635138e+00
1.23087347e-01]

Feature list: Index([

'texture_mean',
'area_mean',
'smoothness_mean',
'concavity_mean',
'symmetry_mean',
'fractal_dimension_mean',
'texture_se',
'area_se',
'smoothness_se',
'concavity_se',
'symmetry_se',
'fractal_dimension_se',
'smoothness_worst',
'concavity_worst',
'symmetry_worst',
'fractal_dimension_worst'],
dtype='object')

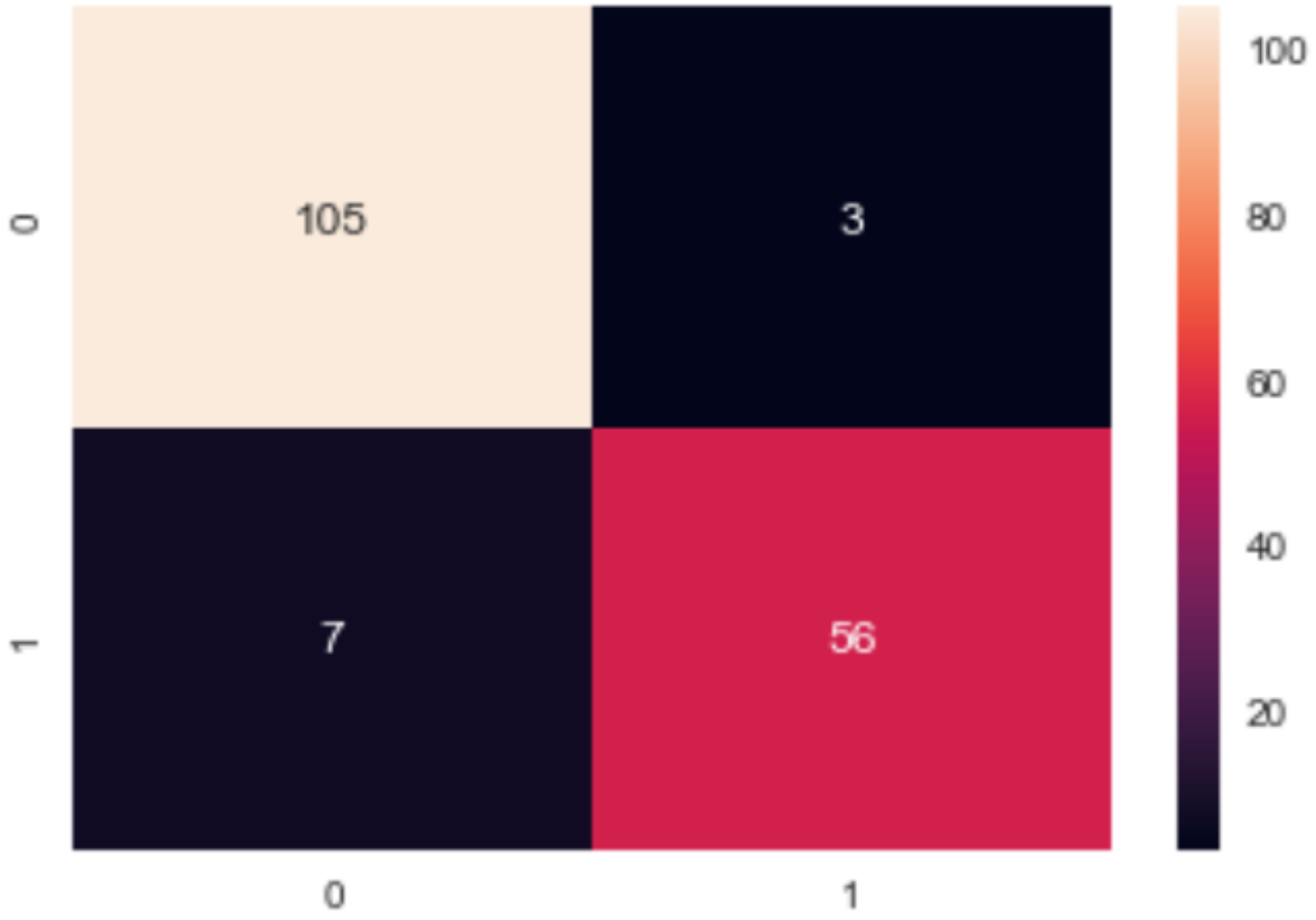
Best 5 feature to classify is that

area_mean,
area_se,
texture_mean,
concavity_worst
concavity_mean.

So lets se what happens
if we use only these
best scored 5 feature.

In [24]:
```
x_train_2 = select_feature.transform(x_train)
x_test_2 = select_feature.transform(x_test)
#random forest classifier with n_estimators=10 (default)
clf_rf_2 = RandomForestClassifier()
clr_rf_2 = clf_rf_2.fit(x_train_2,y_train)
ac_2 = accuracy_score(y_test,clf_rf_2.predict(x_test_2))
print('Accuracy is: ',ac_2)
cm_2 = confusion_matrix(y_test,clf_rf_2.predict(x_test_2))
sns.heatmap(cm_2,annot=True,fmt="d")
```

```
Accuracy is:  0.9415204678362573
```


105
3
7
56
0
1
0
1
100
80
60
40
20

5) Tree based feature selection and random forest classification

http://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html In random forest classification method there is a **feature_importances_** attributes that is the feature importances (the higher, the more important the feature). **!!! To use feature_importance method, in training data there should not be correlated features. Random forest choose randomly at each iteration, therefore sequence of feature importance list can change.**

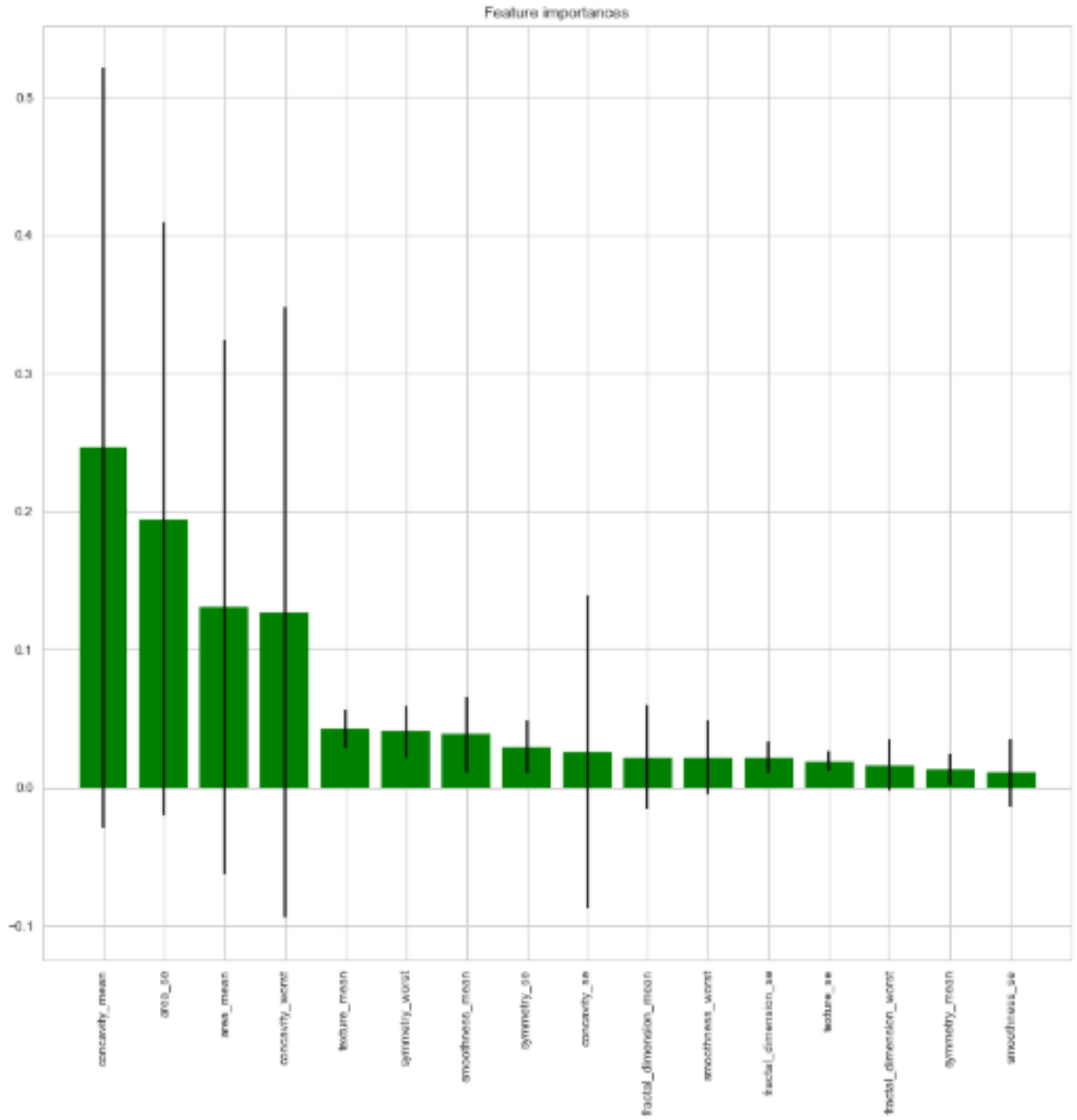
In [29]:
```
clf_rf_5 = RandomForestClassifier()
print(x_train.size,y_train.size,x_train.size/y_train.size)
clr_rf_5 = clf_rf_5.fit(x_train,y_train)
importances = clr_rf_5.feature_importances_
std = np.std([tree.feature_importances_ for tree \
              in clf_rf.estimators_], axis=0)
indices = np.argsort(importances)[::-1]

# Print the feature ranking
print("Feature ranking:")

for f in range(x_train.shape[1]):
    print("%d. feature %d (%f)" % (f + 1, indices[f], \
                                   importances[indices[f]]))
```

```
6368 398 16.0
Feature ranking:
1. feature 3 (0.246238)
2. feature 7 (0.194224)
3. feature 1 (0.130795)
4. feature 13 (0.127333)
5. feature 0 (0.042514)
6. feature 14 (0.040405)
7. feature 2 (0.038522)
8. feature 10 (0.029042)
9. feature 9 (0.025950)
10. feature 5 (0.022152)
11. feature 12 (0.021752)
12. feature 11 (0.021708)
13. feature 6 (0.019282)
14. feature 15 (0.016109)
```

```
# Plot the feature importances of the forest
plt.figure(1, figsize=(14, 13))
plt.title("Feature importances")
plt.bar(range(x_train.shape[1]), importances[indices], \
        color="g", yerr=std[indices], align="center")
plt.xticks(range(x_train.shape[1]), x_train.columns[indices],\
           rotation=90)
plt.xlim([-1, x_train.shape[1]])
plt.show()
```



As you can seen in plot above, after 5 best features importance of features decrease. Therefore we can focus these 5 features. As I sad before, I give importance to understand features and find best of them.

Conclusion

Shortly, I tried to show importance of feature selection and data visualization. Default data includes 33 feature but after feature selection we drop this number from 33 to 5 with accuracy 95%. In this kernel we just tried basic things, I am sure with these data visualization and feature selection methods, you can easily ecxeed the % 95 accuracy. Maybe you can use other classification methods.

Homework - 1:

(1) Change Swarmplot to stripplot
(2) Replace RandomForestClassifier by sequential ANN
(3) Increase accuracy to 98%
(4) Add confusion matrix to Iris codes

Homework - 2:

(5) Add cross validation

(6) Change to: clf_rf = RandomForestClassifier (n_estimators=10, max_depth=8, random_state=3)

Thank you for your attention